

Error Detection

→ Data are transmitted in the network.

→ The data can be corrupted during transmission.

→ Transmission error.

→ Error detection and correction are implemented either at the data link layer or the transport layer of the OSI model.

→ For reliable communication, errors must be detected & corrected.

Types of Error

1) Bit error

2) Burst error.

1) Bit error / or single bit error.

→ only 1 bit in the data unit has been changed.

g. $0001100 \rightarrow 0011100$

2) Burst error

→ 2 or more ~~errors~~ bits change. length of burst error (7 bits)

Sent 0 0 0 1 0 1 1 0 0 1 0
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
0 0 1 0 0 1 0 1 0
Received.

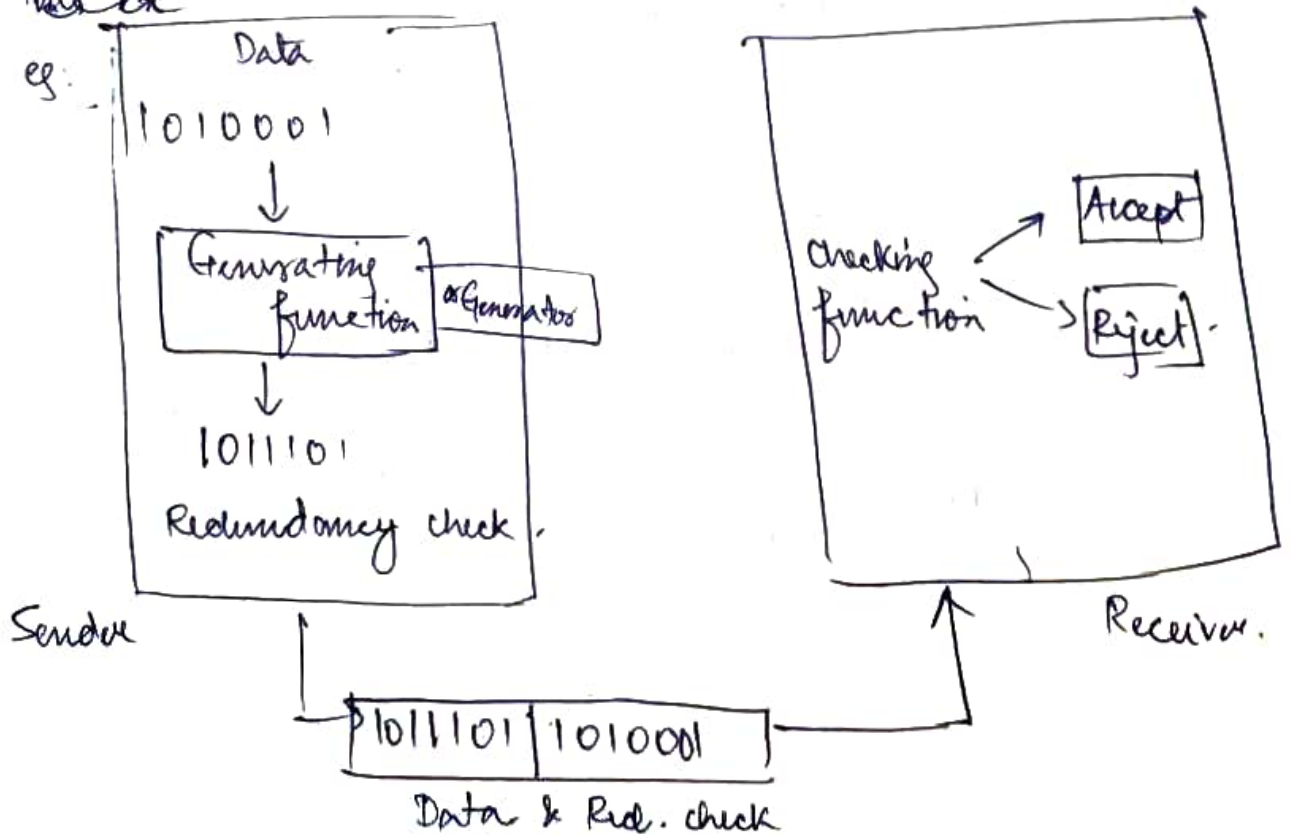
How to detect the errors?

→ Error detection means to decide whether the received data is correct or not without having a copy of the original message.

→ ~~Only using~~ (With only original bits/message, it is difficult to find the errors).

→ To detect / correct errors, we need to send some extra bits with the data (called redundant bits)

→ This ex:



Error Correction.

It can be handled in two ways:-

1) Receiver can have the sender retransmit the entire data unit.

2) The receiver can use an error-correcting code, which automatically corrects certain errors.

Note: Both error detection/correction $\xrightarrow{\text{needs}}$ redundancy
bits/checks

ERROR DETECTION TECHNIQUES

4 types of redundancy checks:-

1) VRC - vertical Redundancy Check or Single Parity Check.

2) LRC - Longitudinal Redundancy "

famous { 3) Checksum.

4) cyclic RC (CRC)

1) VRC or Single Parity Check or Parity Check

Data 1100001

↓
Even Parity
Generator

VRC
bit.

1

11100001

Data transmitted

Receiver

11100001

↓
if even no. of 1
then
no error

Note: VRC : 1 if odd no. of 1s.
0 if even no. of 1s

Performance of VRC

- 1) It can detect single bit error.
- 2) " " " lowest error only if the no. of ^{1s} errors is odd.

eg: 11100001 $\rightarrow$ 10100001 $\rightarrow$ Receiver Reject

11100001 $\rightarrow$ 10100101 $\rightarrow$ Accepts.

or 001001001 $\rightarrow$ Accept (but actually wrong).
(odd 1s)

Q

1110110	1101111	1110010
↓	↓	↓
<u>1</u>	<u>0</u>	<u>0</u>
11101101	11011110	11100100

LRC or Two Dimensional Parity

$\rightarrow$ The parity bit is calculated for each column & sent along with the data.

$\rightarrow$ The block of parity acts as the redundant bits.

Q Find LRC for data blocks 11100111, 11011101, 00111001, 10101001

& determine the data that is transmitted?

→

1	1	1	0	0	1	1	1
1	1	0	1	1	1	0	1
0	0	1	1	1	0	0	1
1	0	1	0	1	0	0	1
LRC → 1 0 1 0 1 0 1 0							

odd no of 1's → 1
even no. of 1's → 0

Data transmitted:

10101010	10101001	00111001	11011101	11100111
LRC	Data			

Received

Performance of LRC

- 1) LRC increases the likelihood of detecting burst errors.
- 2) If two bits in one data units are damaged and two bits in exactly the same positions in another data unit are also damaged, the LRC checker will not detect an error.

Q Find LRC → 0110111 10101001 01101001 10101010.
& data transmitted.

Checksum

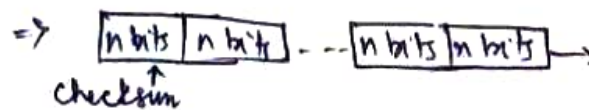
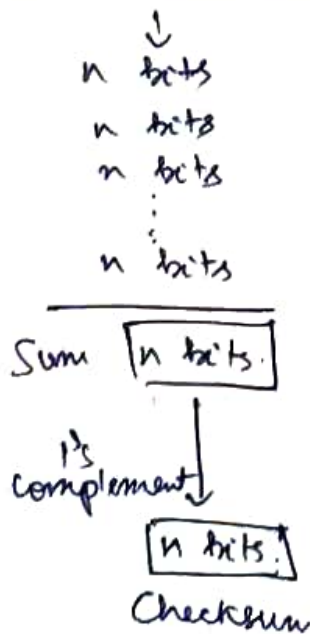
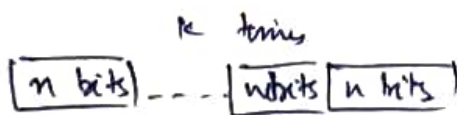
(Bottom up approach)

Sender side $\rightarrow$ Checksum creation.

Receiver side $\rightarrow$ Checksum validation.

Operation At Sender Side.

- 1) Break the original message into 'K' no. of blocks with 'n' bits in each block.
- 2) Sum all the 'K' data blocks.
- 3) Add the carry to the sum (if any)
- 4) Do 1's complement to the sum = Checksum.



All 1s, accept
Otherwise, reject

Divided into 8 bit

eg:-

10011001 11100010 00100100 10000100

Senders

```

1 0 0 1 1 0 0 1
1 1 1 0 0 0 1 0
0 0 1 0 0 1 0 0
1 0 0 0 0 1 0 0
-----
1 0 0 1 0 0 0 1
    
```

→ 10

0 0 1 0 0 1 0 1

1's complement → 11011010 ⇒ Checksum

Checksum | Original data
↓
Send.

Operation at Receiver Side

- Collect all the data blocks including the checksum.
- Sum all the data blocks and checksum.
- If the result is all 1's, ACCEPT, Else, REJECT.

```

1 0 0 1 1 0 0 1
1 1 1 0 0 0 1 0
0 0 1 0 0 1 0 0
1 0 0 0 0 1 0 0
1 1 0 1 1 0 1 0 → Checksum
-----
1 1 1 1 1 1 1 0
    1 0
-----
1 1 1 1 1 1 1 1
    
```

Performance of checksum

- The checksum detects all errors involving an odd nos. of bits.
- It detects most errors involving an even no. of bits.
- If one or more bits of a segment are damaged and the corresponding bit or bits of opp. values in a second segment are also damaged, the sums of those columns will not change and the receiver will not detect the errors.

Cyclic Redundancy Check

Q Find the CRC for the data blocks 100100 with the divisor 1101?

→ CRC generation at Sender Side:-

- 1) Find the length of the divisor 'L'.
- 2) Append 'L-1' bits to the original message.
- 3) Perform binary division operation.
- 4) Remainder of the division = CRC.

Note:-

The CRC must be of L-1 bits.

(*) Divisor is common to both sender & receiver (because protocol makes it)

A	B	XOR
0	0	0
0	1	1
1	0	1
1	1	0

~~For~~ $L=4$; So, 3 0's are appended to the message

$$\begin{array}{r}
 111101 \\
 \hline
 1101 \overline{) 1001000000} \\
 \underline{1101} \\
 010000 \\
 \underline{1101} \\
 1010 \\
 \underline{1101} \\
 1110 \\
 \underline{1101} \\
 1100 \\
 \underline{000} \\
 1100 \\
 \underline{1101} \\
 001
 \end{array}$$

$\therefore \text{CRC} = 001$

Data Transmitted = $100100 \underline{001}_{\text{CRC}}$

Find CRC for 1110010101 with divisor $x^3 + x^2 + 1$

$\rightarrow$ Divisor = 1101.

Original message = 1110010101

$$\begin{array}{r}
 1101 \overline{) 1110010101000} \\
 \underline{1101} \\
 1101 \\
 \underline{1101} \\
 10100 \\
 \underline{1101} \\
 11010 \\
 \underline{1101} \\
 110
 \end{array}$$

$\therefore \text{CRC} = 110$

Data transmitted = $1110010101 \underline{110}_{\text{CRC}}$

Part 2 (Receiver side)

$$\begin{array}{r} 1101 \overline{) 100100001} \end{array}$$

$$\begin{array}{r} 1101 \end{array}$$

$$\begin{array}{r} 1000 \end{array}$$

$$\begin{array}{r} 1101 \end{array}$$

$$\begin{array}{r} 1010 \end{array}$$

$$\begin{array}{r} 1101 \end{array}$$

$$\begin{array}{r} 1110 \end{array}$$

$$\begin{array}{r} 1101 \end{array}$$

$$\begin{array}{r} 1101 \end{array}$$

$$\begin{array}{r} 1101 \end{array}$$

$$\begin{array}{r} 0000 \end{array}$$

(Data accepted - no error).

Q Suppose we want to transmit the message 11001001 and protect it from errors using the CRC polynomial x^3+1 . Use polynomial long division to determine the ~~new~~ message that should be transmitted. Corrupt the left-most third bit of the transmitted message & show that the error is detected by the receiver using CRC technique.

→

GATE (2007)

The message 11001001 is to be transmitted using CRC polynomial x^3+1 to protect it from errors. The message that should be transmitted is:-

→ Divisor = 1001

Orig msg = 11001001

$$\begin{array}{r} 1001 \overline{) 11001001000} \\ \underline{1001} \\ 1011 \\ \underline{1001} \\ 1000 \\ \underline{1001} \\ 1100 \\ \underline{1001} \\ 1010 \\ \underline{1001} \\ 011 \end{array}$$

Divisor = 1001

Orig msg = 11001001 011